

Act as a Principal Digital Twin Architect, Business Systems Architect, Data Engineer, Simulation Systems Engineer, Graph Data Architect, JavaScript Developer, SaaS Security Engineer, and Product Designer.

Your task is to build Layer 4 of an AI Business Decision Intelligence Platform as a standalone HTML application.

The layer is called:

**BUSINESS DIGITAL TWIN LAYER**

This layer depends on:

1. Data Acquisition Layer
2. Business Operations Layer
3. Business Intelligence Layer

The existing Simulation Engine will be connected after this layer is complete.

Do not rewrite the existing simulation engine.

Do not build final AI Decision Intelligence yet.

## **Core Purpose**

Create a continuously updated virtual representation of each real business.

The Digital Twin must answer:

1. What is the current state of the business?
2. How did the business reach this state?
3. How are its components connected?
4. Which constraints and dependencies are active?
5. Which simulation assumptions can be replaced with real business data?
6. How accurately does the twin represent the real business?

## **Required Processing Flow**

Validated operational events → Business Intelligence snapshot → Twin state update → Entity and relationship update → Constraint evaluation → Dependency evaluation → Confidence calculation → Twin version increment → Historical snapshot → Simulation-ready context

## **Existing Architecture Compatibility**

Use:

`window.INFINICUS.bus.register(route, handler, schema)`

`window.INFINICUS.bus.call(route, payload)`

`window.INFINICUS.bus.emit(event, data)`

`window.INFINICUS.bus.on(event, listener)`

Use the namespace:

`twin.*`

Do not create another independent event bus.

## Data Dependency

The Digital Twin must consume canonical intelligence snapshots generated by the Business Intelligence Layer.

Expected event:

`intelligence.snapshot.generated`

Expected source structure:

`{ businessId, generatedAt, currentState, trends, anomalies, healthScore, targets, alerts, dataQuality }`

The twin must not independently recalculate all Business Intelligence metrics.

## Required Modules

Build:

1. Twin Registry
2. Twin identity and versioning
3. Canonical twin state
4. Entity Registry
5. Entity Graph
6. Relationship Engine
7. Financial state model
8. Commercial state model
9. Customer state model
10. Inventory state model
11. Operational state model
12. Workforce state model

13. Supplier state model
14. Market context model
15. Constraint Engine
16. Dependency Engine
17. State Update Processor
18. Historical Snapshot Engine
19. Data Confidence Engine
20. Twin Fidelity Score
21. Assumption Registry
22. Simulation Context Builder
23. Scenario Branching
24. Scenario Comparison
25. Actual-versus-Predicted Reconciliation
26. Twin dashboards
27. Existing Simulation Engine adapter

## Canonical Twin State

Use:

{ twinId, businessId, twinVersion, effectiveAt,

identity, financial, commercial, customers, inventory, operations, workforce, suppliers, market,

constraints, dependencies, assumptions, risks, opportunities,

health, confidence, provenance }

Every major section must preserve:

current value previous value change trend timestamp source confidence calculation version

## Twin Versioning

Increment the twin version after every meaningful accepted update.

Preserve:

previousVersion currentVersion reason sourceSnapshotId changedSections updatedAt

Never overwrite history without retaining version metadata.

## Entity Graph

Support entities:

business branch product service customer employee supplier order inventory item cash register marketing campaign equipment account task loan contract

Support relationships:

BUSINESS\_HAS\_BRANCH BRANCH\_SELLS\_PRODUCT CUSTOMER\_BUYS\_PRODUCT  
EMPLOYEE\_WORKS\_AT\_BRANCH SUPPLIER\_SUPPLIES\_PRODUCT PRODUCT\_CONSUMES\_INVENTORY  
ORDER\_GENERATES\_PAYMENT EXPENSE\_REDUCES\_CASH CAMPAIGN\_ATTRACTS\_CUSTOMER  
TASK\_SUPPORTS\_OPERATION LOAN\_INCREASES\_CASH LOAN\_CREATES\_OBLIGATION

Every relationship must include:

strength confidence evidence count first observed date last observed date

Do not label a relationship as causal without sufficient evidence.

## Constraint Engine

Support:

financial cash capacity inventory workforce supplier regulatory market technology time branch customer

Each constraint must contain:

constraintId type severity entity description currentValue thresholdValue unit status confidence evidence

## Dependency Engine

Identify dependencies between:

suppliers and products products and inventory branches and employees customers and revenue marketing and acquisition loans and cash obligations equipment and production capacity

Each dependency must contain:

source target dependencyType dependencyStrength disruptionImpact confidence evidence

## Data Confidence

Calculate:

completeness freshness consistency accuracy coverage source reliability historical depth

Return:

overall confidence confidence by state section missing data stale data conflicting data low-reliability sources

## **Twin Fidelity Score**

Create a Twin Fidelity Score separate from the Business Health Score.

Business Health measures the condition of the business.

Twin Fidelity measures how accurately and completely the twin represents the business.

Use transparent, documented, versioned weights.

Return:

fidelity score fidelity status simulation readiness section scores missing requirements reliability warnings

## **Assumption Registry**

Register every simulation parameter and identify its source.

Supported sources:

observed\_business\_data calculated\_business\_metric user\_input verified\_external\_data industry\_benchmark  
system\_default synthetic\_estimate

Use this priority:

Observed business data → Calculated business metric → Verified user input → Verified external data →  
Industry benchmark → System default

Never silently replace real data with benchmark assumptions.

## **Simulation Context Builder**

Generate a simulation-ready structure containing:

simulationContextId twinId twinVersion businessId normalized simulation inputs constraints dependencies  
assumptions confidence benchmark versions generatedAt

Every simulation must record:

twinId twinVersion simulationContextId assumption versions benchmark versions simulation engine  
version

## Scenario Branching

Create temporary alternative states without modifying the real twin.

Flow:

Current twin → Clone scenario state → Apply proposed changes → Validate changes → Build simulation context → Run simulation → Compare outcomes → Preserve or discard scenario

Support scenarios such as:

price changes marketing changes hiring reducing staff opening a branch closing a branch adding capital taking a loan changing suppliers changing inventory policy introducing a new product

## Prediction Reconciliation

After a simulated decision is implemented, compare:

predicted result actual result prediction error evaluation period data confidence

Preserve the reconciliation for future Decision Intelligence learning.

## EngineBus Routes

Register explicit routes under:

*twin.registry. twin.state. twin.entity. twin.relationship. twin.constraint. twin.dependency. twin.confidence. twin.fidelity. twin.assumption. twin.snapshot. twin.context. twin.scenario. twin.reconciliation.\**

## Security and Integrity

Implement:

business ownership validation branch isolation twin version integrity schema versioning snapshot immutability safe HTML rendering input sanitization scenario isolation duplicate update prevention source-data traceability confidence validation assumption transparency audit logging error envelopes safe IndexedDB operations storage corruption handling

Do not expose one business's twin to another business.

Do not allow scenarios to modify live operational data.

## User Interface

Use:

black background neon green accents white typography IBM Plex Sans IBM Plex Mono mobile-first responsive design desktop left navigation clear confidence labels visible fidelity status entity and relationship views constraint severity indicators assumption source labels simulation-readiness warnings version history timeline

Desktop navigation:

Twin Overview Business State Entities Relationships Constraints Dependencies Assumptions Scenarios History Readiness Settings

Mobile navigation:

Twin State Constraints Scenarios More

## Storage

Use:

localStorage for preferences IndexedDB for twin states, relationships, snapshots, scenarios, and reconciliations a clean adapter for future Firestore, Supabase, or another cloud database

Use separate structured stores for:

digital twins twin versions twin states entities relationships constraints dependencies assumptions confidence scores fidelity scores snapshots simulation contexts scenario branches scenario changes prediction reconciliations audit logs

Do not store the complete platform only inside one unstructured JSON document.

## Build Method

Build in small executable blocks.

For every block:

1. State the purpose.
2. State dependencies.
3. Provide only the code required for that block.
4. Explain exactly where to insert it.
5. Provide a manual test procedure.
6. Review security, accuracy, state-integrity, and versioning risks.

7. Stop after completing the block.

Do not regenerate the full HTML after every block.

Do not remove or rewrite previous working modules.

## **First Response**

Do not begin coding immediately.

First provide:

1. Digital Twin architecture.
2. Module dependency map.
3. Canonical twin-state schema.
4. Entity and relationship model.
5. Constraint and dependency model.
6. Confidence and fidelity calculation method.
7. Assumption Registry design.
8. Simulation Context contract.
9. Scenario Branching design.
10. EngineBus route map.
11. Event map.
12. Storage architecture.
13. Exact development block sequence.

After the architecture review, begin with Block 1 only.